

Punto 3.2 — Consultas básicas: Where, OrderBy y ThenBy

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.1. Se mantiene la separación Domain/Application/Infrastructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Consultas básicas: Where, OrderBy y ThenBy** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.2
dotnet restore AceriaData.sln
```

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerPendientesPorCliente`, `ObtenerPorEstadoOrdenadasPorFecha`, `ObtenerPorRangoDeFechas`, `ObtenerPorClienteOrdenadas`, `ObtenerPorClienteYRangoDeFechas` y `ObtenerSqlConsultaBasica`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infrastructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/ConsultasBasicasUseCase.cs`

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class ConsultasBasicasUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public ConsultasBasicasUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== WHERE, ORDERBY Y THENBY ===");
        var norte = _unidad.Ordenes.ObtenerPendientesPorCliente("Constructora del Norte");
        var pendientes = _unidad.Ordenes.ObtenerPorEstadoOrdenadasPorFecha("Pendiente");
        var rango = _unidad.Ordenes.ObtenerPorRangoDeFechas(new DateTime(2024,1,1), new DateTime(2024,
12,31));
        var ordenadas = _unidad.Ordenes.ObtenerPorClienteOrdenadas("Constructora del Norte");
        var combinada = _unidad.Ordenes.ObtenerPorClienteYRangoDeFechas("Constructora del Norte", new
DateTime(2024,1,1), new DateTime(2024,12,31));
        if (norte.Count != 2 || pendientes.Count != 3 || rango.Count != 5 || ordenadas.Count != 3 ||
combinada.Count != 3) throw new InvalidOperationException("Resultados de filtros/ordenaciones
inesperados.");
        if (!ordenadas.Select(o => o.NumeroOrden).SequenceEqual(new[] { "0F-2024-0004", "0F-2024-0003",
"0F-2024-0001" })) throw new InvalidOperationException("ThenByDescending no produjo el orden
esperado.");
        Console.WriteLine($"Norte pendientes: {norte.Count} | Pendientes: {pendientes.Count} | Rango:
{rango.Count} | Norte ordenadas: {ordenadas.Count}");
        Console.WriteLine(_unidad.Ordenes.ObtenerSqlConsultaBasica());
    }
}

```

Explicación línea a línea del caso de uso 3.2

Línea 1: `using AceriaData.Application.Interfaces;` → Importa los puertos de Application, en especial IUnidadDeTrabajo e IOrdenRepositorio, que desacoplan el caso de uso de EF Core.

Línea 3: `namespace AceriaData.Application.UseCases;` → Declara el espacio de nombres de la capa a la que pertenece el archivo.

Línea 5: `public sealed class ConsultasBasicasUseCase` → Declara el caso de uso que valida filtros y ordenaciones del punto 3.2.

Línea 6: `{` → Abre el bloque de la clase.

Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de DbContext.

Línea 8: `public ConsultasBasicasUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;` → Recibe la unidad de trabajo mediante inyección de dependencias.

Línea 10: `public void Ejecutar()` → Define la operación que ejecuta todas las comprobaciones E2E de este checkpoint.

Línea 11: `{` → Abre el cuerpo del método Ejecutar.

Línea 12: `Console.WriteLine("=== WHERE, ORDERBY Y THENBY ===");` → Identifica en consola la demostración que se está ejecutando.

Línea 13: `var norte = _unidad.Ordenes.ObtenerPendientesPorCliente("Constructora del Norte");` → Ejecuta el filtro compuesto por cliente y estado y conserva las órdenes pendientes del Norte.

Línea 14: `var pendientes = _unidad.Ordenes.ObtenerPorEstadoOrdenadasPorFecha("Pendiente");` → Obtiene las órdenes pendientes ordenadas por fecha descendente para validar OrderByDescending.

Línea 15: `var rango = _unidad.Ordenes.ObtenerPorRangoDeFechas(new DateTime(2024,1,1), new DateTime(2024,12,31));` → Filtra por intervalo de fechas y valida la composición de comparaciones sobre DateTime.

Línea 16: `var ordenadas = _unidad.Ordenes.ObtenerPorClienteOrdenadas("Constructora del Norte");`
 → Ejecuta la consulta que encadena `OrderBy` y `ThenByDescending` para el cliente indicado.

Línea 17: `var combinada = _unidad.Ordenes.ObtenerPorClienteYRangoDeFechas("Constructora del Norte", new DateTime(2024,1,1), new DateTime(2024,12,31));` → Combina cliente, rango de fechas y ordenación múltiple en una consulta encapsulada.

Línea 18: `if (norte.Count != 2 || pendientes.Count != 3 || rango.Count != 5 || ordenadas.Count != 3 || combinada.Count != 3) throw new InvalidOperationException("Resultados de filtros/ordenaciones inesperados.");` → Comprueba los cardinales deterministas de las cinco consultas y hace fallar el E2E ante cualquier regresión.

Línea 19: `if (!ordenadas.Select(o => o.NumeroOrden).SequenceEqual(new[] { "OF-2024-0004", "OF-2024-0003", "OF-2024-0001" })) throw new InvalidOperationException("ThenByDescending no produjo el orden esperado.");` → Verifica el orden exacto de las tres órdenes del Norte, por lo que no basta con obtener las filas correctas: la ordenación también debe ser correcta.

Línea 20: `Console.WriteLine($"Norte pendientes: {norte.Count} | Pendientes: {pendientes.Count} | Rango: {rango.Count} | Norte ordenadas: {ordenadas.Count}");` → Resume los cardinales validados para que la ejecución sea inspeccionable.

Línea 21: `Console.WriteLine(_unidad.Ordenes.ObtenerSqlConsultaBasica());` → Imprime el SQL generado para la consulta de referencia con filtro y ordenación múltiple.

Línea 22: `}` → Cierra el cuerpo del método `Ejecutar`.

Línea 23: `}` → Cierra la clase `ConsultasBasicasUseCase`.

▮ Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```

using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<ConsultasBasicasUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<ConsultasBasicasUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.2 OK");

```

Explicación línea a línea de Program.cs 3.2

Línea 1: `using AceriaData.Application.UseCases;` → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.

Línea 2: `using AceriaData.ConsoleApp;` → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.

Línea 3: `using AceriaData.Infrastructure;` → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.

Línea 4: `using AceriaData.Infrastructure.Persistence;` → Importa AceriaDbContext para preparar la base de demostración desde el composition root.

Línea 5: `using Microsoft.EntityFrameworkCore;` → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.

Línea 6: `using Microsoft.Extensions.Configuration;` → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.

Línea 7: `using Microsoft.Extensions.DependencyInjection;` → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.

Línea 9: `var configuration = new ConfigurationBuilder()` → Inicia la construcción de la configuración de la aplicación.

Línea 10: `.SetBasePath(AppContext.BaseDirectory)` → Fija como base el directorio de ejecución para localizar appsettings.json.

Línea 11: `.AddJsonFile("appsettings.json", optional: false)` → Carga la configuración JSON obligatoria que contiene la cadena de conexión.

Línea 12: `.AddEnvironmentVariables()` → Permite sobrescribir configuración mediante variables de entorno.

Línea 13: `.Build();` → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer `ConnectionStrings:AceriaDB`.

Línea 15: `var connectionString = configuration.GetConnectionString("AceriaDB")` → Obtiene la cadena `AceriaDB` desde la configuración.

Línea 16: `?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");` → Detiene el arranque con un error explícito si no existe la cadena de conexión `AceriaDB`, evitando ejecutar el ejercicio con una configuración incompleta.

Línea 18: `var services = new ServiceCollection();` → Crea el contenedor de servicios del `composition root`.

Línea 19: `services.AddAceriaInfrastructure(connectionString);` → Registra `DbContext`, repositorio y unidad de trabajo de `Infrastructure`.

Línea 20: `services.AddScoped<ConsultasBasicasUseCase>();` → Registra el caso de uso con ciclo de vida `Scoped` para compartir el mismo ámbito que `DbContext`.

Línea 22: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → Construye el proveedor de servicios y lo declara con `using` para liberar correctamente los servicios `IDisposable` al terminar el proceso.

Línea 23: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 24: `ValidateOnBuild = true,` → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.

Línea 25: `ValidateScopes = true` → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio `Scoped` consumido incorrectamente desde un `Singleton`.

Línea 26: `});` → Cierra las opciones y completa la llamada que construye el proveedor de servicios.

Línea 27: `using var scope = provider.CreateScope();` → Crea el ámbito `Scoped` que compartirá `AceriaDbContext`, `IUnidadDeTrabajo` y el caso de uso durante esta ejecución.

Línea 29: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` para preparar la base de demostración.

Línea 30: `context.Database.EnsureDeleted();` → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.

Línea 31: `context.Database.Migrate();` → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.

Línea 32: `DemoData.Seed(context);` → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.

Línea 33: `context.ChangeTracker.Clear();` → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.

Línea 35: `var useCase = scope.ServiceProvider.GetRequiredService<ConsultasBasicasUseCase>();` → Resuelve el caso de uso concreto registrado para este checkpoint.

Línea 36: `useCase.Ejecutar();` → Ejecuta la demostración y sus aserciones E2E.

Línea 38: `Console.WriteLine("3.2 OK");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.2 OK
```

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: Un segundo `OrderBy` sustituye la ordenación anterior; para añadir criterios se usa `ThenBy`.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Código activable para reproducir el error controlado

Estos bloques ya existen comentados en el checkpoint. Actívalos sólo para reproducir esta variante y restaura después el estado original.

Archivo: `src/AceriaData.Application/ConsultasBasicasUseCase.cs`

Marcador: `ERROR CONTROLADO M03 3.2 - SEGUNDO ORDERBY SUSTITUYE EL PRIMERO`

```
// Demuestra el Paso 9: el segundo OrderBy reemplaza el criterio anterior.
var ordenCorrecto = _unidad.Ordenes.Consulta()
    .Where(o => o.Cliente == "Constructora del Norte")
    .OrderBy(o => o.Estado)
    .ThenByDescending(o => o.FechaCreacion)
    .Select(o => o.NumeroOrden)
    .ToList();

var segundoOrderBy = _unidad.Ordenes.Consulta()
    .Where(o => o.Cliente == "Constructora del Norte")
    .OrderBy(o => o.Estado)
    .OrderByDescending(o => o.FechaCreacion)
    .Select(o => o.NumeroOrden)
    .ToList();

if (ordenCorrecto.SequenceEqual(segundoOrderBy))
    throw new InvalidOperationException("Error controlado 3.2: el segundo OrderBy no mostró la
sustitución esperada.");

Console.WriteLine($"Error controlado 3.2 OK | Correcto: {string.Join(",", ordenCorrecto)} |
Segundo OrderBy: {string.Join(",", segundoOrderBy)}");
```

Comprobación esperada: un segundo `OrderBy` sustituye el orden anterior.

Laboratorio adicional del punto 3.2

Diagnóstico técnico

Prueba filtros por cliente, estado y fechas antes de combinarlos. `ThenBy` añade un criterio a un `OrderBy`; un segundo `OrderBy` sustituiría el criterio anterior. Los métodos de cadena traducibles no deben confundirse con formas necesariamente óptimas para índices.

Reto resuelto y verificación adicional

Para Constructora del Norte durante 2024, valida el método acumulativo que filtra por cliente/rango y ordena por estado ascendente y fecha descendente. El SQL debe parametrizar las fechas y no concatenar valores.

Errores comunes revisados

Error	Causa	Solución
Filtro se aplica en memoria	Se llamó a <code>ToList</code> antes del filtro	Aplicar <code>ToList</code> al final
Ordenación no aplicada	Se llamó a <code>OrderBy</code> después de <code>ToList</code>	Aplicar <code>OrderBy</code> antes de <code>ToList</code>
Múltiples consultas	Se materializó varias veces	Materializar una sola vez al final
<code>IQueryable</code> expuesto en el repositorio	Anti-patrón	Encapsular en métodos específicos (Módulo 4)
Filtro por fecha incorrecto	Se usó <code>==</code> en lugar de <code>>=</code> y <code><=</code>	Usar operadores de comparación
<code>ThenBy</code> sin <code>OrderBy</code>	Se llamó a <code>ThenBy</code> sin <code>OrderBy</code> previo	Llamar a <code>OrderBy</code> primero
Falta el using de EF Core	El método <code>ToQueryString</code> no está disponible	Añadir using <code>Microsoft.EntityFrameworkCore</code> ;

Analogía operativa

Las consultas básicas con LINQ en una acería son como las órdenes de búsqueda que el jefe de planta envía al archivo central. Where es el filtro: "solo quiero las órdenes del cliente Constructora del Norte". OrderBy es el criterio de ordenación: "ordénalas por fecha de creación". ThenBy es el criterio secundario: "y dentro de cada fecha, ordénalas por estado". El archivo central recibe la orden, busca en sus índices y devuelve exactamente lo que se pidió. Si el jefe pidiera todos los documentos y los ordenara él mismo, perdería tiempo y esfuerzo. Eso es lo que pasa cuando se materializa antes de filtrar. Las consultas encapsuladas en el repositorio son como las ventanillas especializadas del archivo: cada una sabe qué buscar y cómo ordenarlo. El jefe no necesita conocer el archivo por dentro: solo pide lo que necesita a la ventanilla correspondiente. Así funcionan las consultas básicas con LINQ: se filtran y se ordenan en el servidor, y los resultados se devuelven ya preparados.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Crea una consulta de un cliente en un rango de fechas ordenada por estado y fecha descendente.

Cómo activar el reto en el checkpoint

Estos bloques ya existen comentados en el checkpoint. Actívalos sólo para reproducir esta variante y restaura después el estado original.

Archivo: `src/AceriaData.Application/Interfaces.cs`

Marcador: RETO M03 3.2 - PUERTO SQL PARAMETRIZADO

```
// Se activa coordinadamente con el repositorio y el caso de uso del reto.
string ObtenerSqlRetoConsultaBasica(string cliente, DateTime desde, DateTime hasta);
```

Archivo: src/AceriaData.Infrastructure/Repositories/Repositories.cs

Marcador: RETO M03 3.2 - SQL PARAMETRIZADO CLIENTE Y RANGO

```
// Conserva la misma composición del método acumulativo y expone ToQueryString
// únicamente como apoyo pedagógico para comprobar parámetros, ORDER BY y DESC.
public string ObtenerSqlRetoConsultaBasica(string cliente, DateTime desde, DateTime hasta) =>
    _context.OrdenesFabricacion
        .Where(o => o.Cliente == cliente && o.FechaCreacion >= desde && o.FechaCreacion <= hasta)
        .OrderBy(o => o.Estado)
        .ThenByDescending(o => o.FechaCreacion)
        .ToQueryString();
```

Archivo: src/AceriaData.Application/ConsultasBasicasUseCase.cs

Marcador: RETO M03 3.2 - CLIENTE RANGO ORDEN Y SQL PARAMETRIZADO

```
// Demuestra el Paso 10 y el laboratorio adicional con Constructora del Norte durante 2024.
var desdeReto = new DateTime(2024, 1, 1);
var hastaReto = new DateTime(2024, 12, 31);
var resultadoReto = _unidad.Ordenes.ObtenerPorClienteYRangoDeFechas(
    "Constructora del Norte", desdeReto, hastaReto);

var numerosReto = resultadoReto.Select(o => o.NumeroOrden).ToArray();
var ordenEsperado = new[] { "OF-2024-0003", "OF-2024-0004", "OF-2024-0001" };

if (!numerosReto.SequenceEqual(ordenEsperado))
    throw new InvalidOperationException("Reto 3.2: el orden por estado y fecha descendente no coincide con el dataset.");

var sqlReto = _unidad.Ordenes.ObtenerSqlRetoConsultaBasica(
    "Constructora del Norte", desdeReto, hastaReto);

Console.WriteLine($"Reto 3.2 OK | Orden: {string.Join(",", numerosReto)}");
Console.WriteLine(sqlReto);
```

Comprobación esperada: consulta cliente+rango con SQL parametrizado y orden estable.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.2 OK, el siguiente estado parte exactamente de esta solución y añade **Proyecciones con Select y tipos anónimos**.